

Day 3: Structural Directives & Routing

SECTIONS :-

- 1) Structural Directives- ngIf-Else, ngFor, ngSwitch
- 2) Routing
- 3) Tasks
- 4) Extra

1) Structural Directives:-

Before we start the structural directives, we first need to know- ngTemplate. `<ng-template>` is a special Angular element used to define a block of HTML that is **not rendered immediately** (Render matlab display, ye term yaad rakh lo web dev mein har jagah aayega). It's special because it's not like `<p>`, `<h1>`, `<table>` tags. These tags, if they are written they're sure to be displayed but `<ng-template>` just gives a template. It can store whatever you want to display and display it according to your condition/requirement. So if you write -

```
<ng-template>
  <p>Hello from template!</p>
</ng-template>
```

Nothing appears on the screen because Angular stores this template but does not render (display) it automatically. The

template will only be displayed when angular explicitly tells it to display.

So this is very useful if-

- You want to display something based on a condition. (This is similar to using `if-else` statements in programming.)
- You want to repeatedly display a block of HTML. (This is similar to using loops in programming.)
- You want to display different content based on different cases.

To implement these concepts in HTML (yes, HTML—not a programming language like TypeScript, JavaScript, or Python), Angular provides special features called **structural directives**.

The three most commonly used structural directives are:

1. `*ngIf`
2. `*ngFor`
3. `*ngSwitch`

1) *ngIf

`*ngIf` is a structural directive used to display an HTML element only when a condition is true.

Syntax:

```
<p *ngIf="isLoggedIn">Welcome User!</p>
```

Suppose in your `.ts` file you have:

```
isLoggedIn = true;
```

Then Angular displays:

Welcome User!

But if:

```
isLoggedIn = false;
```

then nothing is displayed. Angular completely removes the element from the page instead of simply hiding it.

Using an Else Block

Most of the time, you want to display one thing when the condition is true and something else when the condition is false.

For this, Angular uses `ng-template`.

Example:

```
<p *ngIf="isLoggedIn; else loggedOut">
```

```
  Welcome User!
```

```
</p>
```

```
<ng-template #loggedOut>
```

<p>Please Login First</p>

</ng-template>

If `isLoggedIn` is `true`, Angular displays:

Welcome User!

If `isLoggedIn` is `false`, Angular displays:

Please Login First

Notice that the else block is stored inside an `ng-template`. Angular only renders that template when the condition becomes false.

Why Do We Use the * Symbol?

You may wonder why we write:

<p *ngIf="isLoggedIn">

Welcome User!

</p>

instead of:

<p ngIf="isLoggedIn">

Welcome User!

</p>

The reason is that `ngIf` actually works with `ng-template`.

When Angular sees:

```
<p *ngIf="isLoggedIn">
```

```
  Welcome User!
```

```
</p>
```

it automatically converts it into:

```
<ng-template [ngIf]="isLoggedIn">
```

```
  <p>Welcome User!</p>
```

```
</ng-template>
```

This happens behind the scenes.

The `*` is simply a shortcut that tells Angular:

"Create an `ng-template` for me and apply this structural directive to it."

Now let's understand:

```
[ngIf]="isLoggedIn"
```

The square brackets (`[ ]`) represent property binding.

Angular takes the value of `isLoggedIn` from the `.ts` file and passes it to the `ngIf` directive.

If `isLoggedIn` is `true`, Angular displays the template.

If `isLoggedIn` is `false`, Angular does not display the template.

So whenever you write:

```
*ngIf="condition"
```

you can think of it as Angular automatically creating an `ng-template` and applying `[ngIf]="condition"` to it.

2) *ngFor

Suppose you have the following array in your `.ts` file:

```
students = ["Ram", "Shyam", "Mohan"];
```

Now, if you want to display all the student names, you could use `*ngFor`:

```
<p *ngFor="let student of students">  
  {{ student }}
```

</p>

Angular will display:

Ram

Shyam

Mohan

Understanding the Syntax

*ngFor="let student of students"

This can be read as:

"For every item in the students array, store that item in a variable called student and execute the HTML block once."

In the first iteration:

student = "Ram"

In the second iteration:

student = "Shyam"

In the third iteration:

student = "Mohan"

Each time, Angular creates a new `<p>` tag and places the current value inside it.

Using Index

Sometimes you also want to know the position of each item.

For this, Angular provides **index**.

Example:

```
<p *ngFor="let student of students; let i = index">
```

```
  {{ i }} - {{ student }}
```

```
</p>
```

Output:

0 - Ram

1 - Shyam

2 - Mohan

Here:

i = 0

i = 1

i = 2

for each iteration respectively.

3)*ngSwitch :-

Suppose you have the following variable in your **.ts** file:


```
day = "Monday";
```

Now, depending on the value of `day`, you want to display different messages.

You can do this using `ngSwitch`:

```
<div [ngSwitch]="day">

  <p *ngSwitchCase="Monday">
    Start of the work week!
  </p>

  <p *ngSwitchCase="Friday">
    Weekend is near!
  </p>

  <p *ngSwitchDefault>
    Normal day.
  </p>

</div>
```

Since the value of `day` is `"Monday"`, Angular displays:

Start of the work week!

#new things to check- `ng-container`, and `ng-content`

2)ROUTING-

Routing means navigating from one page to another within an Angular application.

To go from one place to another, you need to know the address of the destination. In Angular, this address is called a **route**.

Since an application can have many pages, Angular needs a way to store all these routes together. This is where the concept of a **routing tree** comes in.

You can think of the routing tree as a structure that contains all the routes of the application. For now, just think of it as the place where Angular keeps track of all the routes.

Before learning how to create routes, it is important to understand the concept of the **Router**.

Every Angular application has one Router. The Router is responsible for maintaining the routing tree and navigating between routes.

Angular separates feature-related code from routing-related code. That is why you will often see two files:

- `module.ts` – contains the module configuration.

- `routing.module.ts` – contains the routing configuration.

This separation keeps routing logic independent from the rest of the module logic.

STEPS TO FOLLOW WHEN DOING ROUTING-

1.Setting Up the Routing Module

1. Import `RouterModule` and `Routes`.

```
import { RouterModule, Routes } from  
'@angular/router';
```

2. Import all the components for which you want to directly create routes.

3. Create a routes array.

```
const routes: Routes = [];
```

4. Add route definitions inside the array.

Example:

```
const routes: Routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'login', component: LoginComponent }  
];
```

```
];
```

This routes array becomes part of the application's routing tree.

5. Configure the routing module.

```
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule]  
})
```

`forRoot(routes)` registers these routes with Angular's Router and creates the initial routing tree for the application.

2.Setting Up the Module-

Import the routing module into your module.

Example:

```
imports: [  
  AppRoutingModule  
]
```

This makes the routing configuration available to the module.

3. Setting up [Component.ts](#) file-

In the component where you want to perform navigation, import the Router class.

```
import { Router } from '@angular/router';
```

Inject it through the constructor.

```
constructor(private router: Router) {}
```

4. Use the routes-

You can then navigate programmatically using:

```
this.router.navigate(['grp-a/member1']);
```

This tells Angular to navigate to the route:

```
/grp-a/member1
```

The Router looks for this route in the routing tree and loads the corresponding component.

❖ **Important Note-**

If a component uses routing directives such as `routerLink` or `router-outlet`, then `RouterModule` must be available in the module that declares that component.

Usually, this happens automatically when the corresponding `routing.module.ts` file is imported into the module. If `RouterModule` is not available, Angular will not recognize directives such as `routerLink` and `router-outlet`.

Once you are comfortable with setting up routes using the steps above, the next thing to learn is the different levels of routing.

There are three levels of routing that you should understand:

1) Simple Sibling Routing

This is the most basic form of routing.

In sibling routing, all routes exist at the same level in the routing tree.

Example:

`/home`

/login
/about
/contact

Each route directly loads its corresponding component. There is no parent-child relationship between the routes.

2) Nested Routing

In nested routing, some routes exist inside other routes. This creates a parent-child relationship in the routing tree.

Example:

/grp-a
 /member1
 /member2
 /member3

Here, **grp-a** is the parent route and **member1**, **member2**, and **member3** are child routes.

Nested routing is useful when multiple pages belong to the same section of the application and should share a common layout.

3) Lazy Loading

In all the examples so far, Angular loads the components as soon as the application starts.

This is fine for small applications, but large applications may contain hundreds of components.

With lazy loading, Angular loads a module only when the user actually visits its route.

Example:

/admin

The Admin module is not loaded when the application starts. It is loaded only when the user navigates to the `/admin` route.

This improves performance because Angular does not have to load the entire application at once.

You can think of lazy loading as loading routes and modules only when they are needed.

Routing Practice Roadmap

Whenever you are given a routing problem, first identify which level it belongs to:

- Are all pages at the same level? → Sibling Routing
 - Does one page contain child pages? → Nested Routing
 - Should a module be loaded only when needed? → Lazy Loading
-

Level 1: Simple Sibling Routing

Steps

1. Read the problem and list all the pages that need to be created.
2. Treat every page as independent.
3. Give each page its own route.
4. Place all routes at the same level in the routes array.
5. Create navigation links or buttons between the pages.
6. Test each route by directly typing its URL.
7. Verify that navigating to one page completely replaces the previous page.

Task 1

Create the following pages:

/home

/about

/contact

/help

Create buttons that allow navigation between all pages.

Task 2

Create a Student Management application.

Routes:

/dashboard

/students

/teachers

/courses

Add a navbar and allow navigation between all pages.

Task 3

Create an E-Commerce application.

Routes:

/home

/products

/cart

/profile

/orders

Add navigation links and verify that every page loads correctly.

Level 2: Nested Routing

Steps

1. Read the problem and identify the parent page.
2. Identify all pages that belong under that parent.
3. Create a parent route.
4. Create child routes inside the parent route.
5. Add a router-outlet inside the parent component.
6. Create buttons or links inside the parent component to navigate between child routes.
7. Verify that the parent page remains visible while only the child section changes.

Task 1

Create:

/department

 /cse

 /ece

 /mechanical

The Department page should contain buttons for all departments.

Task 2

Create:

/company

 /hr

 /finance

 /development

 /testing

The Company page should remain visible while switching between child pages.

Task 3

Create:

```
/grp-a  
  /member1  
  /member2  
  /member3
```

Display member information inside the Group A page using nested routing.

Level 3: Lazy Loading

Steps

1. Read the problem and identify the feature/module that can be separated from the rest of the application.
2. Create a dedicated module for that feature.
3. Move all related components into that module.
4. Create routes inside that module.
5. Connect the module to the main routing tree using lazy loading.
6. Ensure the feature module is not directly imported into the main module.
7. Verify that the feature loads only when its route is visited.

Task 1

Create an Admin module.

Routes:

/admin

/admin/users

/admin/settings

Configure the Admin module using lazy loading.

Task 2

Create a Student module.

Routes:

/student

/student/profile

/student/attendance

/student/results

Configure the Student module using lazy loading.

Task 3

Create three separate modules:

/admin

/teacher

/student

Each module should contain at least two child pages.

Configure all three modules using lazy loading and verify that they load independently when visited.

4)Extra-

Logging using console.log and debugging in Inspect was mentioned today. Please check it on your own.

THANK YOU.